



U.S. DEPARTMENT
of ENERGY



BERKELEY LAB



IRI Technical Subcommittee Proposed changes for v2



John MacAuley
ESnet

Weekly Meeting
July 30, 2026

Evolving Requirements

- Original v1 design goals focused on simplified schema for consumption by low-code end users.
- New requirements driven by AmSC resource orchestration have asked for a more descriptive data model with dynamically queryable system/user state and topology relationships.
- Agentic workflows introduce the need for separation of static descriptive data from dynamic state information to better fit the RAG versus MCP model.

With the v2 focus of shifting to more complex system interaction, the original driving requirement for a simple API will need to be relaxed in favor of a more consumable capability.

URN-based object types

- V1 resource type definitions are too broad to provide any determination of capability and was primarily defined for use to drive display on a status web page
 - Defined as an enumeration, requiring an OpenAPI schema modification any time a new resource type was introduced.
- Intelligent applications wish to dynamically discover facility capabilities so need more specific resource concepts such as GPUs, scratch filesystems, object storage, Globus DTN services, XRootD services, and inference services
 - The primary goal is to provide a stable, hierarchical identifier for resource and service type values without requiring the OpenAPI schema to be revised whenever a new subtype is introduced.
 - Separates the stability of the data model from the evolution of the type taxonomy.

Solution: Replace enum-based types properties with a structured URN representing a more specific type, and a registry process for defining IRI-type URNs.

```
{
  "id": "orion",
  "name": "Orion",
  "description": "Lustre HPE ClusterStor system",
  "resource_type": "urn:doe-iri:resource:storage:system",
  "group": "production-storage",
  "status": "up",
  "last_modified": "2026-01-22T12:00:00Z",
  "supported_endpoints": [
    "filesystem"
  ],
  "self_uri": "https://api.example.org/api/v2/status/resources/orion",
  "site_uri": "https://api.example.org/api/v2/facility/sites/example-site",
  "capability_uris": []
}
```

Type-specific attributes for objects

- V1 resource objects have a fixed set of attributes for all resource types
 - Adding any new properties to the resource definition would need to apply to all resource types.
- Facilities also need to describe information that is meaningful only for certain classes of resources. Examples include:
 - Filesystem capacity and storage technology;
 - Compute-node architecture, accelerator inventory, and queue associations;
 - Data-transfer protocol support and endpoint characteristics;
 - Service implementation details and operational capabilities.
- Adding every such property to the common Resource schema would create a broad, unstable, and difficult-to-implement model.

Solution: Define an optional attributes property as the designated extension point for type-specific resource metadata.

```
{
  "id": "orion",
  "name": "Orion",
  "description": "Lustre HPE ClusterStor system",
  "resource_type": "urn:doe-iri:resource:storage:system",
  "group": "production-storage",
  "status": "up",
  "last_modified": "2026-01-22T12:00:00Z",
  "supported_endpoints": [
    "filesystem"
  ],
  "self_uri": "https://api.example.org/api/v2/status/resources/orion",
  "site_uri": "https://api.example.org/api/v2/facility/sites/example-site",
  "capability_uris": [],
  "attributes": {
    "schema_version": "1.0.0",
    "storage_type": "urn:doe-iri:storage:filesystem",
    "filesystem_technology": "lustre",
    "vendor": "HPE",
    "product": "ClusterStor"
  }
}
```

(Re)Introducing HAL _links

- V1 resource objects have a fixed set of relationships for all resource types, with no flexible way to add new per resource type relationships.
- URL to related functionality (storage device/filesystem API) are not discoverable, and require knowledge to construct.
- HAL **_links** is a JSON/XML convention for adding standardized hypermedia links
 - Clients discover available resources and navigation paths from the response rather than constructing every URL themselves.
- A **_links** property exposes navigable relationships between the current resource and related API resources
 - Each link is identified by a relation name, such as self, iri:site, iri:status, iri:provides-filesystem, iri:depends-on.
 - The href value provides the target URI, allowing clients to follow links instead of constructing URLs.
 - Links may include metadata such as URI templates, media types, titles, or deprecation information.
- This mechanism supports runtime API discovery and HATEOAS-style navigation.
- AI systems understand and can reason over **_links**
 - Each relation URI should resolve to documentation describing the link semantics.
 - See RFC8288 “Web Linking” for more information.

```
{
  "id": "orion",
  "name": "Orion",
  "description": "Lustre HPE ClusterStor system",
  "resource_type": "urn:doe-iri:resource:storage:system",
  "group": "production-storage",
  "status": "up",
  "last_modified": "2026-01-22T12:00:00Z",
  "self_uri": "https://api.example.org/api/v2/status/resources/orion",
  "site_uri": "https://api.example.org/api/v2/facility/sites/example-site",
  "_links": {
    "curies": [
      {
        "name": "iri",
        "href": "https://api.example.org/rels/{rel}",
        "templated": true
      }
    ],
    "self": {
      "href": "https://api.example.org/api/v2/status/resources/orion"
    },
    "iri:site": {
      "href": "https://api.example.org/api/v2/facility/sites/example-site"
    },
    "iri:provides-filesystem": {
      "href": "https://api.example.org/api/v2/status/resources/orion-scratch"
    },
    "iri:incidents": {
      "href": "https://api.example.org/api/v2/status/incidents?resource_id=orion"
    },
    "iri:connected-to": [
      { "href": "https://api.example.org/api/v2/status/resources/dtn-01" },
      { "href": "https://api.example.org/api/v2/status/resources/frontier" }
    ]
  }
}
```

The Resource Card

- Within the American Science Cloud framework a Resource Card:
 - Is a standardized metadata profile, similar in spirit to a Model Card or Dataset Card from ModCon.
 - Registers and describes a specific scientific asset so that both human researchers and automated AI agents can discover, access, and orchestrate it across the federated ecosystem.
- A resource registered via a Resource Card can be:
 - **Computational Hardware:** HPC clusters, GPU nodes, or edge inference accelerators.
 - **Scientific Datasets & Storage:** Curated experimental data, simulation outputs, or high-performance file systems.
 - **AI Models & Frameworks:** Pre-trained scientific foundation models, surrogate models, or domain-specific neural networks developed under ModCon.
 - **Experimental Instruments & Facilities:** Automated lab equipment, light sources (like APS), or beamlines capable of remote or agent-driven steering.
- Why It Matters
 - Users of AmSC are increasingly relying on agentic workflows autonomously designing experiments, querying data, and launching jobs.
 - Resource Cards provide the machine-readable "blueprints" that allow the AmSC orchestrator to locate available compute resources, move data efficiently, and run complex scientific workflows automatically across national facilities.

Incorporating the Resource Card concept into IRI

- A single Resource object that combines definition and state creates several problems:
 - **AI retrieval inefficiency:** AI systems must dynamically retrieve resource information even when the needed facts are stable.
 - **Poor cache behavior:** A representation containing volatile state has a short useful cache lifetime even if most of the object is stable.
 - **Ambiguous freshness:** Consumers cannot easily determine whether a field is a durable fact about the resource or a live operational observation.
 - **Schema pressure:** Adding every static and dynamic property to one resource model creates a large and unstable schema.
 - **Unnecessary MCP calls:** MCP tools may be invoked to rediscover static facts that could have been retrieved from an index.
 - **RAG contamination:** Volatile state values may be indexed and later returned as if they were still current.
 - **Weak relationship modeling:** Static relationships such as connected-to, provides-filesystem, or has-direct-connection are different from dynamic state values, but a single mixed object does not make that distinction clear.

Solution: Separate resource definition from resource state by making the *resource definition durable and link-oriented*, while keeping *live state behind dynamic endpoints*. This separation allows stable resource descriptions to be indexed once and reused, while volatile state is retrieved only when needed.

Separating Resource definition from Resource state

- Separate the Resource object into two related representations:
 - **ResourceDefinition**
 - **ResourceState**
- A **ResourceDefinition** provides a stable resource card that can be indexed by a RAG system or loaded into an AI planning context without requiring a live call to every facility API during each query.
- A **ResourceState** provides current or recent operational information and is retrieved dynamically through links published in the **ResourceDefinition**.
- The design is especially useful for MCP-enabled AI systems
 - The AI system can retrieve stable resource definitions from an index, reason over those definitions, and invoke MCP tools only when the user's question requires the current state.
- The **ResourceDefinition** object is intentionally congruent to the American Science Cloud Resource Card concept.

The current /status/resources Resource definition is similar in concept to ResourceState and could perform that role for backwards compatibility.

Design Goals

- Separate stable resource meaning from volatile operational state.
- Make **ResourceDefinition** suitable for RAG indexing and AI planning.
- Make **ResourceDefinition** information suitable for reuse in AmSC Resource Cards.
- Avoid indexing dynamic values that may become stale.
- Reduce unnecessary MCP and API calls for static resource facts.
- Preserve dynamic retrieval for values that require freshness.
- Provide clear client guidance about when to call live APIs.
- Use HAL-style `_links` for relationships and dynamic retrieval targets.
- Support explicit resource graph relationships in **ResourceDefinition** when they are infrequently changing.
- Maintain compatibility with existing Resource concepts where practical.
- Keep the common resource schema small and extensible.
- Allow type-specific attributes without mixing them with live state (both **ResourceDefinition** and **ResourceState** can have type-specific attributes).
- Provide stable identifiers that support cross-facility AI workflows.
- Improve cacheability and update semantics.
- Enable better provenance and freshness reasoning.

Conceptual Model

- A **ResourceDefinition** SHOULD be safe to store in a knowledge base or RAG index.
- A **ResourceState** SHOULD be treated as time-sensitive and SHOULD NOT be indexed as durable knowledge unless the index preserves timestamp, source, and freshness semantics.

ResourceDefinition

- stable identity
- stable classification
- stable or slowly changing attributes
- stable relationships
- links to dynamic state

ResourceState

- current status
- current availability
- current capacity
- current health
- active incidents
- recent events
- state observation timestamp

Property Classification Rules

- A property SHOULD be placed in **ResourceDefinition** when it satisfies one or more of the following conditions:
 - It describes the identity of the resource.
 - It describes the class or type of the resource.
 - It describes stable capabilities or supported interfaces.
 - It describes stable relationships to other resources.
 - It changes only through administrative or configuration action.
 - It is useful for discovery, planning, documentation, or AI reasoning.
 - It can be cached or indexed without materially misleading consumers.
- A property SHOULD be placed in **ResourceState** or another linked dynamic representation when it satisfies one or more of the following conditions:
 - It describes the current operational condition of the resource.
 - It changes frequently.
 - It is time-sensitive.
 - Its correctness depends on when it was observed.
 - It represents availability, utilization, health, events, incidents, or capacity remaining.
 - It should be fetched only when the client needs current information.
 - It should not be indexed as durable knowledge.

ResourceDefinition Example

```
{
  "id": "orion",
  "name": "Orion",
  "description": "Lustre HPE ClusterStor system",
  "resource_type": "urn:doe-iri:resource:storage:system",
  "group": "production-storage",
  "supported_endpoints": [
    "filesystem"
  ],
  "site_uri": "https://api.example.org/api/v2/sites/example-site",
  "capability_uris": [],
  "attributes": {
    "schema_version": "1.0.0",
    "storage_type": "urn:doe-iri:storage:filesystem",
    "filesystem_type": "lustre",
    "vendor": "HPE",
    "product": "ClusterStor",
    "capacity": {
      "total_bytes": "67900000000000000"
    }
  }
},
```

```
  "_links": {
    "curies": [
      {
        "name": "iri",
        "href": "https://iri.science/rels/{rel}",
        "templated": true
      }
    ],
    "self": {
      "href": "https://api.example.org/api/v2/resource-definitions/orion"
    },
    "iri:state": {
      "href": "https://api.example.org/api/v2/status/resources/orion"
    },
    "iri:capacity": {
      "href": "https://api.example.org/api/v2/status/resources/orion/capacity"
    },
    "iri:events": {
      "href": "https://api.example.org/api/v2/status/events?resource_id=orion"
    },
    "iri:incidents": {
      "href": "https://api.example.org/api/v2/status/incidents?resource_id=orion"
    },
    "iri:provides-filesystem": {
      "href": "https://api.example.org/api/v2/resource-definitions/orion-scratch"
    },
    "iri:connected-to": [
      {
        "href": "https://api.example.org/api/v2/resource-definitions/dtn-01"
      },
      {
        "href": "https://api.example.org/api/v2/resource-definitions/frontier"
      }
    ]
  }
}
```

ResourceState Example

```
{
  "id": "orion-status",
  "resource_id": "orion",
  "observed_at": "2026-07-20T17:00:00Z",
  "state_valid_until": "2026-07-20T17:05:00Z",
  "polling_frequency_seconds": "600",
  "resource_type": "urn:doe-iri:resource:storage:system",
  "status": "up",
  "health": {
    "summary": "operational"
  },
  "attributes": {
    "schema_version": "1.0.0",
    "capacity": {
      "total_bytes": "67900000000000000",
      "available_bytes": "6000000000000000",
      "used_bytes": "7900000000000000"
    }
  },
}
```

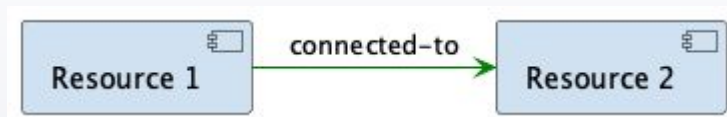
```
  "_links": {
    "curies": [
      {
        "name": "iri",
        "href": "https://iri.science/rels/{rel}",
        "templated": true
      }
    ],
    "self": {
      "href": "https://api.example.org/api/v2/status/resources/orion"
    },
    "iri:definition": {
      "href": "https://api.example.org/api/v2/resource-definitions/orion"
    },
    "iri:events": {
      "href": "https://api.example.org/api/v2/status/events?resource_id=orion"
    },
    "iri:incidents": {
      "href": "https://api.example.org/api/v2/status/incidents?resource_id=orion"
    }
  }
}
```

The current /status/resources Resource definition is similar in concept to ResourceState and could perform that role for backwards compatibility.

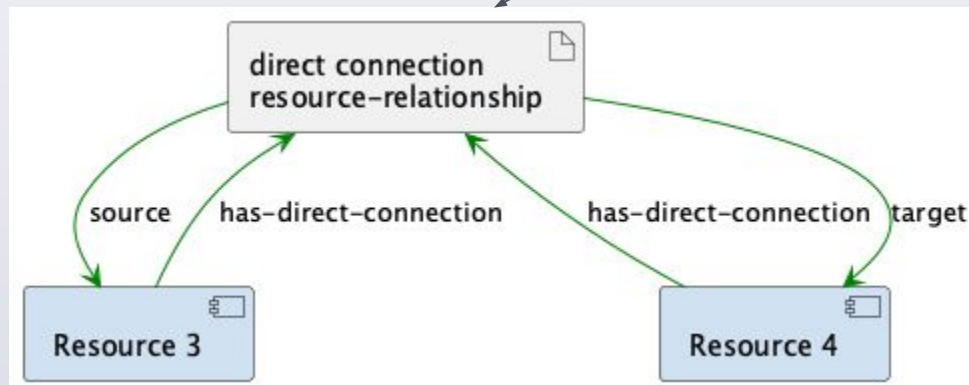
Physical and Logical Topology

- Workflow systems, portals, notebooks, agents, data movers, and automation frameworks need to understand where data can be stored, where jobs can run, how filesystems are mounted, and which transfer services can access those filesystems.
- Existing facility APIs can describe resources, status, allocations, jobs, and filesystem operations, but workflow automation also needs a topology view that answers questions such as:
 - Which compute systems can access a given filesystem?
 - Which storage system backs a given filesystem?
 - Which DTN services export a given filesystem?
 - Which project directories, home directories, scratch areas, queues, and accounts are available to this authenticated user?
 - Which physical resources are unavailable, degraded, or indirectly impacted by a dependency?
 - Which effective paths should a workflow use for staging, execution, and result transfer?
- Physical Topology
 - Describes the facility-owned resource graph, independent of any specific user.
 - It outlines the resources that exist and the operational relationships among them.
- Logical Topology
 - Describes the effective resource view available to an authenticated user.
 - It is derived by combining the physical topology (the facility-owned resource graph), with user-specific and policy-driven information.

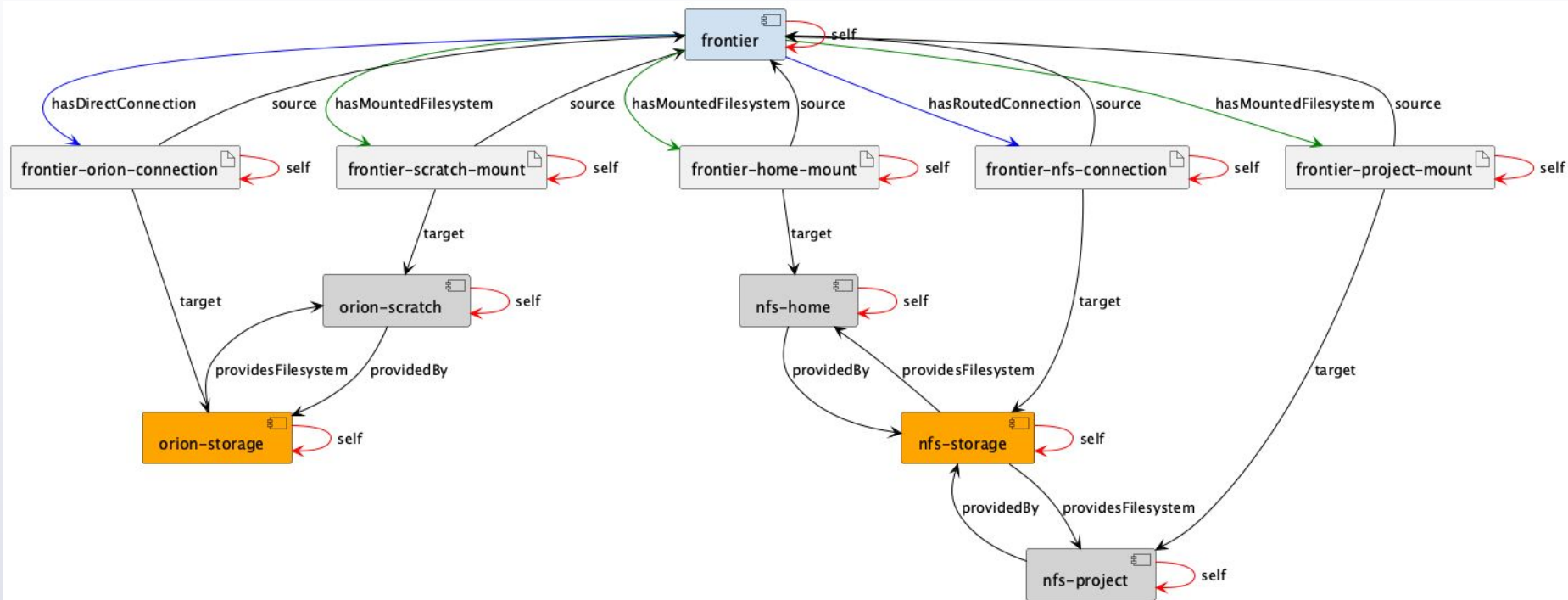
Concepts



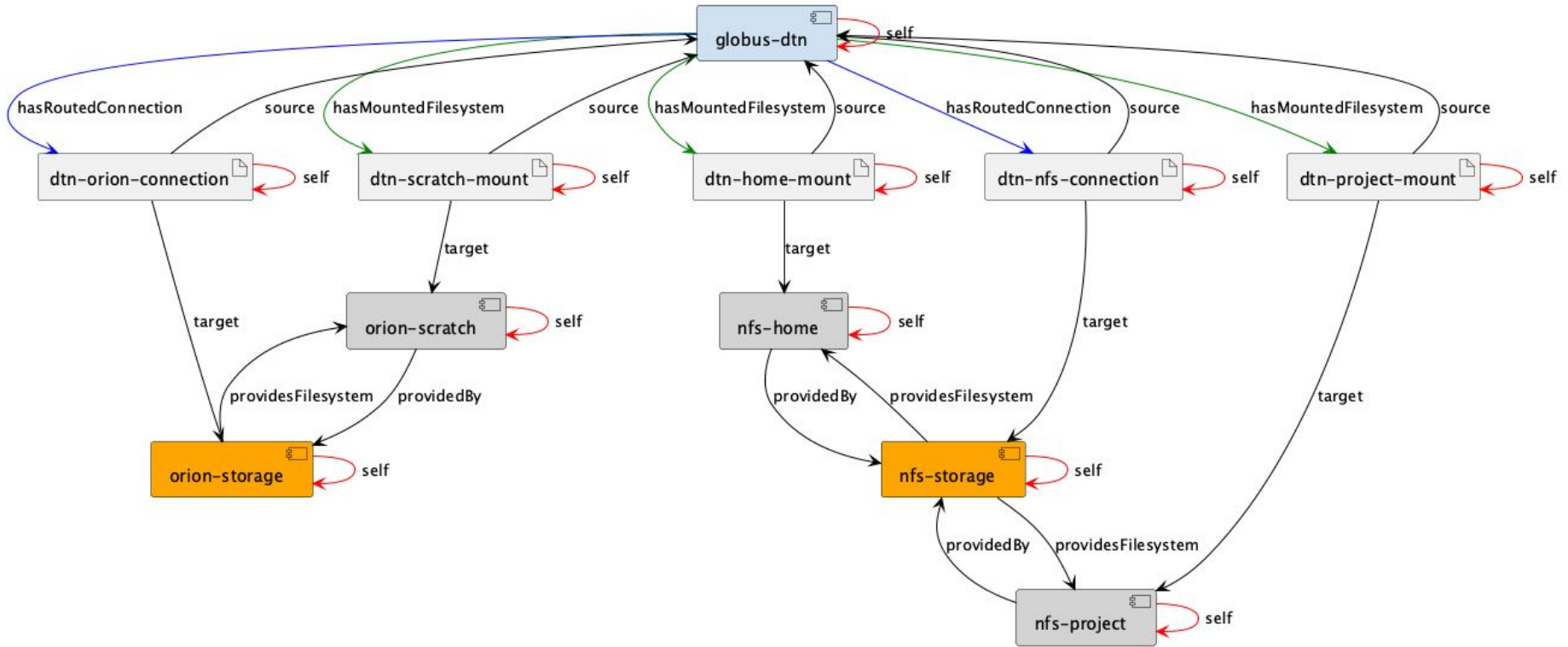
```
{  
  "id": "infiniband-relationship-3-4"  
  "source": "resource-3",  
  "target": "resource-4",  
  "relationship_type": "urn:doe-iri:relationship:networking:infiniband"  
  "attributes": {  
    "connection_type": "urn:doe-iri:networking:infiniband:xdr",  
    "data_rate_gbps": "800"  
  },  
  "_links": {  
    "self":  
      "iri:status"  
  }  
}
```



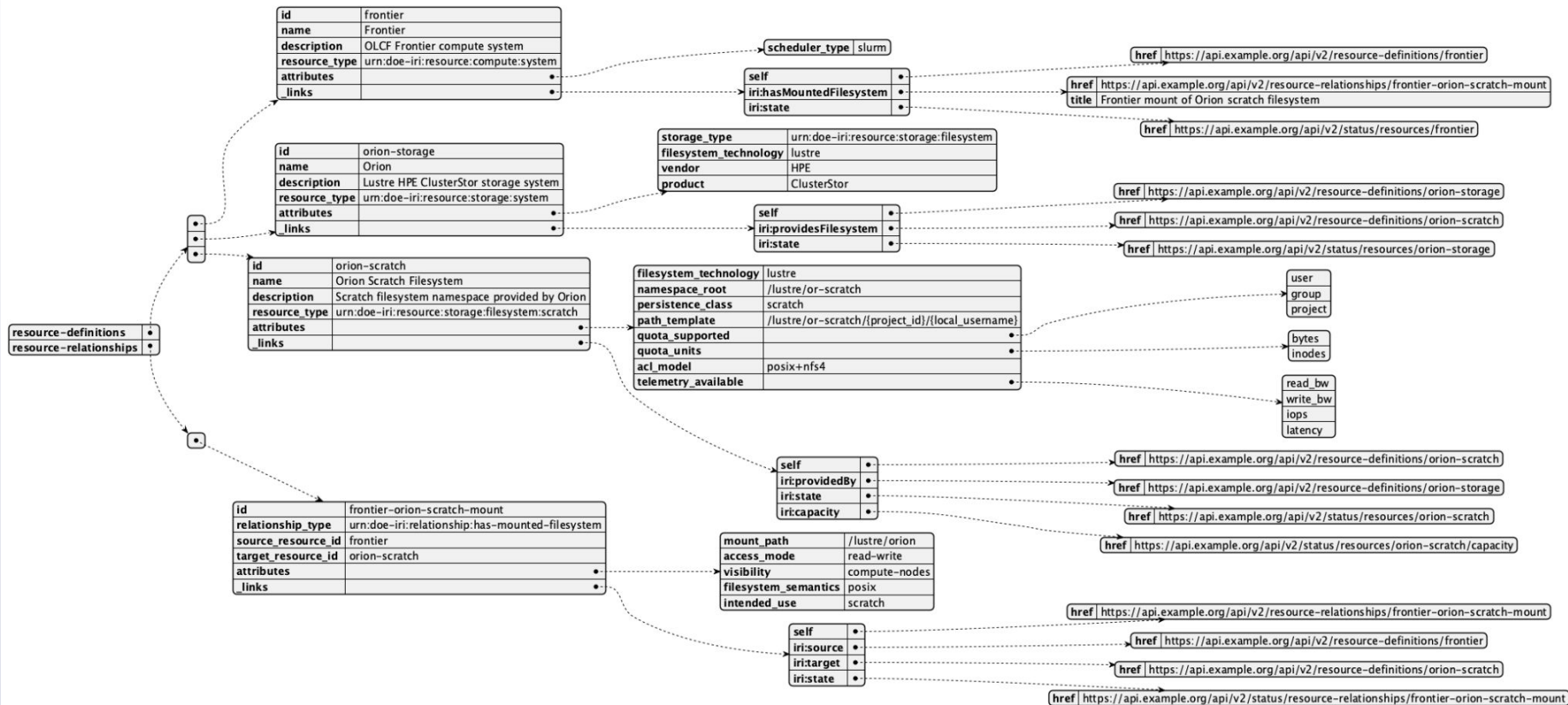
Physical Topology - Frontier



Physical Topology - Globus DTN



Physical Topology



The background is a solid blue color with a network of white lines and dots. The dots are of various sizes and are connected by thin white lines, creating a web-like pattern. The lines and dots are more concentrated in the upper half of the image, with a few scattered dots and lines in the lower half. The overall effect is a sense of connectivity and technology.

Fin